

UNIVERSITY OF THESSALY

DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING

School of Science · Undergraduate Programme

ADVANCED DATA MANAGEMENT

Final Assignment

Academic Year 2025 – 2026 · Spring Semester

Weight: 100 % of final grade

Individual or pairs (3 students max) · Duration: Final Day of Examination

1. Assessment Overview

1.1 Purpose

This assessment asks you to do more than apply techniques you have rehearsed in class. It asks you to reason about trade-offs, defend choices under constraints that pull in different directions, and produce a prototype that a second engineer could run and inspect within minutes. The scenario you will work with — a smart power grid analytics platform — was chosen because it sits at the intersection of systems thinking, data engineering, and electrical infrastructure, three areas central to your degree.

The platform requires you to combine every major data technology covered during the semester: relational storage with JSON extensions, in-memory caching, document-oriented storage, column-family databases, and graph databases. No single technology handles every problem well here, and that is intentional. Part of what this assessment measures is your ability to recognise where a tool helps and where forcing it would hurt.

1.2 What You Will Demonstrate

By submitting this work you should be able to demonstrate:

- A grounded understanding of why distributed data systems make the trade-offs they do, traced from first principles rather than memorised from slides.
- The ability to design schemas and query patterns across five different storage technologies without conflating their semantics.
- Practical skills in containerised deployment, API development, and data modelling that translate directly to an engineering job.
- The habit of measuring what you build and being honest about what the measurements actually show.
- A reflective voice: the ability to look at your own decisions and articulate, clearly and without defensiveness, what you would do differently with hindsight.

1.3 Bloom's Taxonomy Mapping

Part	Cognitive Level	Primary Activity
A — Critical Thinking	Analyze / Evaluate	Examine trade-offs, argue for or against a position with evidence
B — Implementation	Apply / Create	Design schemas, write code, build a running containerised system
C — Measurement	Evaluate	Design experiments, collect data, interpret results honestly
D — Reflection	Evaluate / Create	Write reasoned arguments; defend personal stances on design choices

1.4 Deliverables

1. A written report as a PDF (20 – 35 pages, excluding appendices and code listings). Body font: Calibri 11 pt, line spacing 1.15, margins 2.5 cm.

2. A GitHub repository with a meaningful commit history. A single final commit is treated as evidence of absent process and is penalised.
3. A docker-compose.yml that boots the entire system with one command: `docker compose up --build`. No manual steps beyond that should be needed.
4. A README.md covering: environment requirements, startup instructions, a brief description of each service, and at least three example API calls (curl or HTTPie).
5. An AI Use Disclosure appendix if any AI tool was used. State what prompts were given and what parts of the submission came from the output. Disclosure does not penalise you; concealment does.

1.5 Academic Integrity

Sources may be cited; they may not be paraphrased without attribution or reproduced without quotation marks. Answers that restate lecture slides in different words, with no personal analysis added, will receive zero on the reasoning sub-criteria regardless of technical correctness. Your marker is reading for evidence of thought, not evidence of memory.

1.6 Grading Weights

Part	Topic	Weight
A	Critical thinking — reasoning and argumentation	30 %
B	Implementation — design, code, and system integration	45 %
C	Measurement — experimental method and interpretation	15 %
D	Reflection — reflection essays and oral questioning	10 %
Bonus	Monitoring stack, CI/CD pipeline, or production-hardening	+ 5 %

2. Scenario: GridSense — Smart Power Grid Analytics & Fault Management

The Regional Power Authority (RPA) operates an electrical distribution network covering a metropolitan area of 1.2 million residents. The network includes 4,800 substations, 22,000 distribution transformers, 180,000 smart meters installed at residential and commercial premises, and approximately 3.4 million sensors measuring voltage, current, power factor, and temperature at intervals of one second to one minute depending on sensor class.

The RPA currently runs three separate legacy systems — a billing database, a SCADA historian for time-series sensor data, and a paper-based fault register — that cannot communicate with each other in real time. Engineers have been told that the next grid failure event must be diagnosed and contained within four minutes of onset. The current average time to diagnosis is twenty-two minutes, largely because no single engineer has access to all the relevant information simultaneously.

Your task is to design and build GridSense: a prototype platform that consolidates these sources, supports real-time monitoring, and provides the analytical capabilities the RPA engineering team needs to reduce that diagnosis time.

2.1 Functional Requirements

1. Ingest and store time-series sensor readings from smart meters and field sensors. Expected write rate: 40,000 events/second at normal load, 120,000 events/second during storm events.
2. Represent the power network topology as a connected graph of substations, transformers, feeders, and endpoints, so that the impact of a fault at any node can be traced downstream in real time.
3. Store equipment profiles (transformers, switchgear, smart meters) with flexible, heterogeneous metadata — manufacturers differ; field format varies per equipment generation.
4. Maintain an account and billing record for each consumer premise, subject to ACID guarantees. Late or incorrect billing generates regulatory penalties.
5. Provide a caching layer for frequently requested dashboard views: current network load by district, top-ten overloaded transformers, active fault alerts.
6. Expose a REST API consumed by engineering workstations, a mobile field app, and a public-facing outage map.
7. Support a fault event log that records every automatic protective relay operation, including the exact sequence of events, for post-incident forensic analysis.

2.2 Non-Functional Requirements

- **Latency:** Dashboard reads < 100 ms at P95. Sensor writes acknowledged < 50 ms. Graph traversal for fault propagation analysis < 200 ms for paths up to depth 6.
- **Availability:** Sensor ingestion and fault alerting must remain operational during any single-node failure. Billing may tolerate brief degraded access but not data loss.

- **Consistency:** Billing records require strong consistency. Sensor counters and dashboard aggregates may lag up to 30 seconds. Fault event sequences must be stored in strict causal order.
- **Scale:** The time-series store must handle at least 200 GB of sensor data per day and retain 90 days of rolling history before archival.
- **Observability:** Every API endpoint must emit request count, error rate, and latency percentiles. (Bonus: integrate Prometheus and Grafana.)

2.3 Proposed System Architecture

The following table describes the five storage roles and their intended technology assignments. You may challenge any of these assignments in Part A, but your implementation in Part B must reflect a justified final decision.

Storage Role	Assigned Technology	Primary Justification
Time-series sensor readings	Apache Cassandra 4.1	Write-heavy, partition by sensor_id + time, linear scale
Network topology graph	Neo4j 5 (Community)	First-class relationship model; Cypher traversal for fault paths
Equipment metadata catalog	MongoDB 7	Schema variability across equipment generations and manufacturers
Consumer billing & accounts	PostgreSQL 15 (JSONB)	ACID transactions; tariff rules stored as JSONB; GIN indexing
Dashboard cache & alerts	Redis 7	Sub-millisecond reads; TTL-based expiry; Pub/Sub for fault alerts

Why This Scenario Works as an Assessment

Each storage technology is required by a genuine constraint in this problem, not inserted artificially. Time-series at 40k events/second eliminates PostgreSQL as the primary sensor store. Fault propagation up to depth 6 across 26,000 network nodes eliminates Cassandra and MongoDB. Equipment metadata with per-generation schema variation gives MongoDB a purpose that no fixed-schema system can serve as naturally. Billing with penalties for incorrect records eliminates eventually consistent stores. Dashboard caching with 30-second staleness tolerance is a textbook Redis problem.

Students who recognise these constraints and articulate them will demonstrate a level of engineering judgment that matters far more to a future employer than knowing the syntax of any individual tool.

PART A • Critical Thinking (30 %)

These questions do not have a single correct answer. They have better and worse answers. Better answers take a clear position, support it with evidence or reasoning, acknowledge the strongest objection to that position, and do not hide in vague generalities. Each sub-question states explicitly what a marker will look for.

A.1 • Why Relational Databases Were Not Enough (5 %)

A senior engineer at the RPA, who has spent his career with Oracle and PostgreSQL, argues: "We already have a relational database. With the right indexes, partitioning, and a time-series extension, we can store everything there. Moving to five different systems will create operational chaos and training debt that this organisation cannot afford."

Address the following:

- **a)** Identify the specific technical argument in his position that has merit and explain precisely why it has merit. Do not dismiss it.
- **b)** Identify where the argument breaks down for the GridSense requirements as stated. Use at least one concrete calculation or benchmark figure to support your point — do not argue purely from analogy.
- **c)** Under what conditions would a PostgreSQL-only design be the correct engineering decision? Be specific about the workload numbers and organisational constraints that would shift the balance.

What the marker looks for in A.1

- The concession in (a) must be genuine. "He has a point about operational simplicity" is acceptable; it is not acceptable to say he is entirely wrong.
- The calculation in (b) might be as simple as: $40,000 \text{ inserts/sec} \times 100 \text{ bytes} = 4 \text{ MB/s}$ sustained write throughput; then consider the implications for WAL, VACUUM, and page fragmentation in a heap-based store over 24 hours. Numbers need not be exact, but they must exist.
- Part (c) prevents the lazy answer. A student who argues that NoSQL is always right has not understood the course.

A.2 • The CAP Theorem Applied (7 %)

During a design review, two engineers disagree about the Cassandra configuration for sensor ingestion. Engineer A wants `write_concern = ONE` and `read_concern = ONE` for maximum throughput. Engineer B says this violates the consistency requirements and insists on `QUORUM` for both.

- **a)** Using the CAP theorem precisely — as stated by Gilbert and Lynch (2002), not the common simplified version — explain what trade-off is actually being made here. Identify where partition tolerance fits into the argument.
- **b)** The replication factor is $RF = 3$, deployed across two data centres. Calculate the minimum consistency levels for reads and writes such that a read always sees the most recent write. Show your working.

- **c)** For the sensor ingestion use case specifically, evaluate whether Engineer A or Engineer B is making the better engineering decision. Your answer must reference the stated latency and consistency requirements from Section 2.2, not abstract CAP theory.
- **d)** The PACELC theorem (Abadi, 2012) adds a dimension that CAP does not capture. Explain that dimension and use it to compare Cassandra and HBase in the context of a 24/7 grid monitoring system where network partitions are rare but latency spikes have direct operational cost.

A.3 • Column-Family Data Modelling (6 %)

You are designing the Cassandra table for sensor readings. The two most common access patterns are: (1) retrieve the last N readings for a given sensor within a time window, and (2) retrieve all sensor readings across the entire network in the last 60 seconds for a live dashboard.

- **a)** Design a table schema that optimises for pattern (1). Define the partition key, clustering key, and explain the physical implications of your choice on data distribution and read efficiency.
- **b)** Explain concretely why pattern (2) cannot be served efficiently by a Cassandra table optimised for pattern (1). What query would you have to write, and why does Cassandra execute it inefficiently?
- **c)** The query-driven design principle says: design one table per query. Propose a second table or a materialised view that makes pattern (2) efficient, and analyse the write amplification this introduces. What happens to your total write throughput?
- **d)** A partition key of sensor_id alone would create hot partitions for sensors that report every second over months. Propose a partition key strategy that distributes load more evenly while preserving the ability to query by sensor within a time range.

A.4 • Graph Databases and the Network Topology Problem (7 %)

The most critical feature of GridSense is fault propagation analysis: when a protective relay trips at node X, the platform must instantly identify all downstream nodes that will lose power and all upstream paths that could restore supply via alternative routing.

- **a)** Explain why this problem is naturally suited to a graph database rather than a relational or document database. Your answer must include a concrete comparison: write the SQL query for a two-hop downstream traversal and the equivalent Cypher query. Comment on the difference in complexity as path depth increases.
- **b)** Define the property graph model for the GridSense network. Specify at minimum: node labels, relationship types, and the properties that belong on each. Justify why certain properties belong on a relationship rather than on a node.
- **c)** Neo4j is a CP system under the CAP framework in its clustered configuration. What does this mean in practice for a fault event that arrives during a leader election? How would you design the system so that this brief window of unavailability does not cause missed fault alerts?
- **d)** Graph databases are sometimes proposed for storing time-series data alongside topology ("store every sensor reading as a node connected to its equipment node"). Evaluate this

proposal critically. Under what data volume does it remain reasonable, and where does it collapse?

Common mistake to avoid in A.4

Answering (a) with "graphs are better because they model relationships" is not an answer — it is a tautology. The marker wants to see the SQL JOIN count climb as you go from depth 2 to depth 6, and to see the Cypher pattern stay constant. That contrast is the argument.

A.5 • Technology Selection Reasoning (5 %)

For each of the five cases below, select one technology from the stack and give a 3–5 sentence justification. Justification must reference the specific access pattern or constraint that drives the choice. Answers of the form "X is good for this kind of data" without explaining why the access pattern demands X will receive no marks.

1. Store the complete sequence of relay operations during a fault event (causal order matters; the log must be immutable once written; forensic queries will replay events chronologically).
2. Find all consumers who share a distribution transformer with a flagged high-consumption meter, within two network hops.
3. Cache the current overload status of each of the 22,000 transformers with a five-second expiry.
4. Store a new type of smart meter whose manufacturer provides 40 non-standard telemetry fields not present in any other meter model.
5. Process monthly billing calculations that apply progressive tariff bands, regulatory surcharges, and time-of-use pricing adjustments to 1.2 million accounts atomically at month end.

PART B • Implementation (45 %)

Part B is assessed on your working system, not on your descriptions of it. A marker will clone your repository, run `docker compose up --build`, and attempt to exercise each endpoint listed in Section B.7. Code that cannot be run is not assessed as implementation. Write your report as if the marker can and will check every claim you make.

B.1 • Required Services

Container Name	Technology	Port	Purpose
api	FastAPI (Python 3.11+)	8000	REST gateway; all business logic
timeseries-db	Apache Cassandra 4.1	9042	Sensor reading storage
graph-db	Neo4j 5 Community	7474 / 7687	Network topology & traversal
catalog-db	MongoDB 7	27017	Equipment metadata
billing-db	PostgreSQL 15	5432	Consumer accounts & billing (JSONB)
cache	Redis 7 Alpine	6379	Dashboard cache; Pub/Sub alerts

All passwords and connection strings must be supplied via a `.env` file. No credentials may appear in `docker-compose.yml` or in committed code. Deduct five percent automatically if this rule is broken.

B.2 • Starter `docker-compose.yml` Fragment

The fragment below provides the Cassandra and Neo4j service definitions as a starting point. You are responsible for adding the remaining four services, defining persistent volumes, a shared backend network, and inter-service health dependencies.

YAML

```
version: "3.9"

services:

  timeseries-db:
    image: cassandra:4.1
    container_name: gridsense_cassandra
    restart: unless-stopped
    ports:
      - "9042:9042"
    environment:
      CASSANDRA_CLUSTER_NAME: "GridSenseCluster"
      CASSANDRA_DC: "dc1"
      HEAP_NEWSIZE: "128M"
      MAX_HEAP_SIZE: "512M"
    volumes:
      - cassandra_data:/var/lib/cassandra
      - ./cql/init.cql:/docker-entrypoint-initdb.d/init.cql
    healthcheck:
```

```

test: ["CMD-SHELL", "nodetool status | grep -q '^UN'"]
interval: 30s
timeout: 10s
retries: 10

graph-db:
image: neo4j:5-community
container_name: gridsense_neo4j
restart: unless-stopped
ports:
- "7474:7474" # Browser UI
- "7687:7687" # Bolt protocol
environment:
NEO4J_AUTH: "neo4j/${NEO4J_PASSWORD}"
NEO4J_PLUGINS: '["apoc"]'
NEO4J_dbms_memory_heap_max__size: "512M"
volumes:
- neo4j_data:/data
- ./neo4j/import:/import
healthcheck:
test: ["CMD", "wget", "-O", "/dev/null", "http://localhost:7474"]
interval: 20s
timeout: 5s
retries: 8

# — remaining services: add catalog-db, billing-db, cache, api —

volumes:
cassandra_data:
neo4j_data:
# add: mongo_data, pg_data, redis_data

networks:
default:
name: gridsense_net

```

B.3 • Cassandra Schema Design

Your CQL initialisation file (./cql/init.cql) must create the keyspace and at minimum the following two tables. You should extend this schema with any additional tables your chosen access patterns require.

CQL

-- cql/init.cql (starter — you must complete and extend this)

```

CREATE KEYSPACE IF NOT EXISTS gridsense
WITH replication = {
  'class': 'SimpleStrategy',
  'replication_factor': 1 -- use 3 in production; 1 is fine for dev
};

```

USE gridsense;

-- Table 1: Optimised for query pattern (1) — readings per sensor over time

```

CREATE TABLE IF NOT EXISTS sensor_readings (
  sensor_id TEXT,
  reading_time TIMESTAMP,
  metric_type TEXT, -- 'voltage', 'current', 'power_factor', 'temp'
  value FLOAT,
  unit TEXT,
  quality_flag TINYINT, -- 0=good, 1=suspect, 2=bad
  PRIMARY KEY ((sensor_id), reading_time)
) WITH CLUSTERING ORDER BY (reading_time DESC)
AND default_time_to_live = 7776000; -- 90 days

-- Table 2: relay operation log — append-only, causal order preserved
CREATE TABLE IF NOT EXISTS relay_events (
  feeder_id TEXT,
  event_time TIMEUUID, -- TIMEUUID embeds causal timestamp
  relay_id TEXT,
  event_type TEXT, -- 'TRIP', 'RECLOSE', 'LOCKOUT'
  fault_type TEXT,
  current_kA FLOAT,
  PRIMARY KEY ((feeder_id), event_time)
) WITH CLUSTERING ORDER BY (event_time ASC);

-- TODO: add a second sensor_readings table or materialized view
-- optimised for the cross-network dashboard query (pattern 2)

```

B.4 • Neo4j Graph Model & Seed Data

Your import script (`./neo4j/import/seed.cypher`) must create a representative subgraph of at least 20 nodes covering the following labels and relationship types. The numbers below are for the seed; your report must describe how the model scales to the full 26,000-node network.

CYPHER

```

// neo4j/import/seed.cypher
// Constraint — run once at startup
CREATE CONSTRAINT substation_id IF NOT EXISTS
  FOR (s:Substation) REQUIRE s.substation_id IS UNIQUE;
CREATE CONSTRAINT transformer_id IF NOT EXISTS
  FOR (t:Transformer) REQUIRE t.asset_id IS UNIQUE;

// — Nodes —————
// Grid Supply Point (top of hierarchy)
CREATE (:GridSupplyPoint {
  gsp_id: "GSP_NORTH", name: "Northern Grid Supply Point",
  voltage_kV: 132, region: "North Metro"
});

// Substations
CREATE (:Substation {
  substation_id: "SS_001", name: "Volos Primary",
  voltage_kV: 11, lat: 39.358, lon: 22.938,
  commissioned_year: 1998
});

```

```

// Distribution Transformers
CREATE (:Transformer {
  asset_id: "TX_001_A", rating_kVA: 400,
  manufacturer: "ABB", model: "ONAN-400",
  installed: date("2012-06-15"),
  last_inspection: date("2024-09-01")
});

// Smart Meters (a few representative ones)
CREATE (:SmartMeter { meter_id: "SM_00001", premise_id: "PREM_10001",
  tariff_class: "residential", phase: "single" });

// — Relationships —————
// MATCH before MERGE to avoid duplicates in production scripts
MATCH (g:GridSupplyPoint {gsp_id:"GSP_NORTH"})
MATCH (s:Substation {substation_id:"SS_001"})
CREATE (g)-[:FEEDS {feeder_id:"F_001", voltage_kV:11, length_km:2.4}]->(s);

MATCH (s:Substation {substation_id:"SS_001"})
MATCH (t:Transformer {asset_id:"TX_001_A"})
CREATE (s)-[:SUPPLIES {cable_id:"CB_001", distance_m:320}]->(t);

MATCH (t:Transformer {asset_id:"TX_001_A"})
MATCH (m:SmartMeter {meter_id:"SM_00001"})
CREATE (t)-[:CONNECTS_TO]->(m);

// — Sample traversal queries (test these work) —————

// Find all nodes downstream of a substation (up to depth 4)
// MATCH (s:Substation {substation_id:'SS_001'})-[:FEEDS|SUPPLIES|CONNECTS_TO*1..4]->(n)
// RETURN labels(n), n LIMIT 50;

// Find meters sharing a transformer with a given meter
// MATCH (:SmartMeter {meter_id:'SM_00001'})<-[:CONNECTS_TO]-(t:Transformer)-[:CONNECTS_TO]->(peer)
// RETURN peer.meter_id, peer.premise_id;

```

B.5 • FastAPI Application — Required Structure

Your API package must be structured as follows. The marker will look for this layout. Diverge if you have a good reason, but document the reason.

```

api/
├── main.py      # FastAPI app factory; lifespan events for DB connections
├── routers/
│   ├── sensors.py # /sensors endpoints (Cassandra)
│   ├── grid.py   # /grid endpoints (Neo4j)
│   ├── equipment.py # /equipment endpoints (MongoDB)
│   ├── billing.py # /billing endpoints (PostgreSQL)
│   └── alerts.py  # /alerts endpoints (Redis Pub/Sub)
├── models/
│   ├── cassandra.py # Pydantic models for sensor data
│   ├── graph.py     # Pydantic models for graph responses
│   └── mongo.py     # Pydantic models for equipment

```

```

|   └─ postgres.py # Pydantic models for billing
└─ db/
|   └─ cassandra.py # async session factory (cassandra-driver)
|   └─ neo4j.py    # async driver factory (neo4j[async])
|   └─ mongo.py    # Motor AsyncIO MotorClient
|   └─ postgres.py # asyncpg pool
|   └─ redis.py    # aioredis connection
└─ Dockerfile
└─ requirements.txt

```

B.6 • Sample Endpoint: Fault Propagation Query

The endpoint below shows the expected pattern for a Neo4j-backed graph traversal. Note the use of async context managers for session management and Pydantic for the response schema.

PYTHON

```

# api/routers/grid.py (excerpt)
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import List
from db.neo4j import get_driver

router = APIRouter(prefix="/grid", tags=["Grid Topology"])

class AffectedNode(BaseModel):
    node_id: str
    node_type: str
    name: str
    depth: int

class FaultImpactResponse(BaseModel):
    origin_id: str
    affected_nodes: List[AffectedNode]
    total_affected: int

@router.get("/fault-impact/{node_id}", response_model=FaultImpactResponse)
async def get_fault_impact(node_id: str, max_depth: int = 6):
    """
    Return all nodes that would lose supply if node_id trips.
    Uses variable-length Cypher traversal — depth is bounded to prevent
    accidental full-graph scans on malformed input.
    """
    if max_depth > 10:
        raise HTTPException(status_code=400,
                            detail="max_depth cannot exceed 10 to protect query performance")

    cypher = """
    MATCH (origin {node_id: $node_id})
    MATCH (origin)-[:FEEDS|SUPPLIES|CONNECTS_TO*1..$depth]->(downstream)
    RETURN labels(downstream)[0] AS node_type,
           downstream.node_id AS node_id,
           downstream.name AS name,
           length(

```

```

        shortestPath((origin)-[:FEEDS|SUPPLIES|CONNECTS_TO*]-(downstream))
    ) AS depth
    ORDER BY depth
    """
    driver = get_driver()
    async with driver.session(database="neo4j") as session:
        result = await session.run(cypher,
                                   node_id=node_id, depth=max_depth)
        records = await result.data()

    if not records and not await _node_exists(driver, node_id):
        raise HTTPException(status_code=404,
                            detail=f'Node '{node_id}' not found in topology graph")

    affected = [AffectedNode(**r) for r in records]
    return FaultImpactResponse(
        origin_id=node_id,
        affected_nodes=affected,
        total_affected=len(affected)
    )

```

B.7 • Mandatory REST Endpoints

Method	Path	Backend	Description
POST	/sensors/readings	Cassandra	Ingest one or a batch of sensor readings
GET	/sensors/{sensor_id}/readings	Cassandra	Last N readings for a sensor (query param: limit, from_time)
GET	/sensors/{sensor_id}/summary	Redis (cache)	Cached latest reading + 1h stats; TTL = 30 s
GET	/grid/fault-impact/{node_id}	Neo4j	Downstream nodes affected by a fault at node_id
GET	/grid/restore-paths/{node_id}	Neo4j	Alternative supply paths to restore a faulted node
POST	/grid/nodes	Neo4j	Add a new node to the topology graph
POST	/grid/relationships	Neo4j	Add a new relationship (feeder, cable, connection)
GET	/equipment/{asset_id}	MongoDB	Full metadata for a single piece of equipment
POST	/equipment	MongoDB	Register new equipment (flexible BSON document)
PATCH	/equipment/{asset_id}	MongoDB	Update fields on existing equipment
GET	/billing/account/{premise_id}	PostgreSQL	Account details + current balance
POST	/billing/invoice	PostgreSQL	Generate monthly invoice (ACID transaction)
GET	/alerts/active	Redis	All active fault alerts from Pub/Sub channel
POST	/alerts/publish	Redis	Publish a new fault alert to subscribers

B.8 • Data Seeding Requirements

1. At least 10 substations, 40 transformers, 200 smart meters, and the relationships between them in Neo4j.

2. At least 50,000 sensor readings spread across 20 distinct sensor IDs in Cassandra (a simple Python script generating random but plausible values is fine).
3. At least 30 equipment records across at least 3 different equipment types in MongoDB, each with a different schema shape.
4. At least 100 consumer accounts in PostgreSQL with sample invoice records.
5. The seeding script must be runnable as a single command: `python scripts/seed.py`. It must be idempotent — running it twice must not create duplicate records.

PART C • Measurement (15 %)

The goal of this section is not to produce impressive numbers. It is to produce honest ones, with enough context that a reader can evaluate whether the numbers mean what you say they mean. Each experiment must state: what was measured, how it was measured, what variables were controlled, and what confounds might be present.

C.1 • Cassandra Write Throughput vs Consistency Level

Using a load-testing tool of your choice (Locust, k6, or a custom Python script using the cassandra-driver bulk writer), measure the sustained write throughput to your sensor_readings table at three consistency levels: ONE, LOCAL_QUORUM, and ALL. Use a single-node cluster (as in development).

Report: events/second at each level, p50 and p95 write latency, and any observed errors. Then answer:

- Why does throughput change between consistency levels even on a single-node cluster? (Hint: think about what ALL requires vs ONE on a single replica.)
- If you were to move to a three-node cluster with RF = 3, how would your expectation for the ALL vs QUORUM difference change, and why?
- At what consistency level would you configure the production sensor ingestion pipeline, and what is your explicit reasoning?

C.2 • Graph Traversal Depth vs Latency

Using your seeded Neo4j graph, measure the median and P95 latency of the /grid/fault-impact/{node_id} endpoint as you vary max_depth from 1 to 8. Run at least 30 iterations per depth level. Use a node with at least 20 downstream nodes for this test.

Report your results in a table and a line chart. Then answer:

- At what depth does latency first cross your 200 ms SLA? How does this inform the default max_depth value you would set in production?
- How does the latency growth curve (linear, polynomial, exponential?) and what does that tell you about the underlying traversal algorithm?
- Your seeded graph has at most a few hundred nodes. Extrapolate honestly: how confident are you that this behaviour generalises to 26,000 nodes? What would change?

C.3 • Redis Cache Effectiveness on Dashboard Endpoints

Configure the /sensors/{sensor_id}/summary endpoint with a 30-second cache TTL backed by Redis. Send 1,000 requests to this endpoint: 500 against a warmed cache, 500 against a cold cache (TTL expired between batches). Record p50, p95, p99 latency and cache hit rate.

Then answer:

- What fraction of the latency difference between cache hit and cache miss is attributable to network overhead vs database query time? How do you know?

- Your cache uses allkeys-lru as the eviction policy. Would a different policy produce better hit rates for this access pattern? Defend your answer with reference to the actual distribution of requests in your test.

C.4 • Schema Flexibility in MongoDB vs PostgreSQL JSONB

Insert the same 30 equipment records into both MongoDB and into a PostgreSQL table with a metadata JSONB column. Then run the following three queries against both:

1. Find all equipment where metadata contains a field named firmware_version with a value starting with '3.'.
2. Find all equipment of type 'SmartMeter' with a rated_voltage field greater than 230.
3. Return a count of records grouped by the value of the type field.

Measure query execution time for each. Report results in a table (mean of 10 runs each). Discuss: for this data volume and query pattern, does the difference matter? At what scale would it start to matter?

PART D • Reflection (10 %)

Each response should be 300 – 500 words. The marker is not looking for a correct answer. They are looking for a person who thinks carefully, changes their mind when evidence demands it, and can tell the difference between a reasoned position and a comfortable one.

D.1 • The Hardest Decision You Made

Describe the single most difficult technical decision you made during this project — one where you genuinely were not sure what the right answer was. Walk through your reasoning as it evolved. What information would have helped you decide faster? What would you do differently?

D.2 • Where the Polyglot Architecture Gets Expensive

Running five different database systems introduces costs that are easy to ignore in a prototype but become significant in production: operational overhead, cross-system transactions, data synchronisation, and team knowledge. Identify two specific places in GridSense where the polyglot design creates a genuine engineering burden. What would you consider doing to reduce that burden, and what would you give up by doing so?

D.3 • Graphs in the Wild

Before this course, how did you think about connected data? How has your thinking changed after working with Neo4j and Cypher? Give one concrete example of a problem in your field of electrical and computer engineering — outside this scenario — where you now believe a graph database would be a better tool than what is typically used. Why?

3. Grading Rubric

3.1 Performance Tiers

Grade Range	What it Looks Like
9.0 – 10	The work shows genuine intellectual ownership. Decisions are justified, not assumed. Measurements are honest. The reflection essays take positions the author is willing to defend. The system runs.
7.5 – 8.9	Solid work with most requirements met. Some design decisions are correct but under-explained. Benchmarks are present but may lack depth of interpretation. System runs with minor issues.
6.0 – 7.4	Functional at a basic level. Several architectural choices are made without adequate justification. Benchmarks may be missing or misinterpreted. Reflection essays stay at a surface level.
5.0 – 5.9	Partially functional. Major components missing or non-operational. Reasoning in Part A relies heavily on paraphrasing lecture material. Measurements absent or invalid.
Below 5.0	Non-functional system, plagiarism, or structural absence of critical thinking. A report without a bibliography, or one whose Part A answers could be generated by searching the slide notes, fails regardless of the code.

3.2 Criteria That Cost Marks

- No .env file; credentials hard-coded or committed: automatic –5 %.
- Single git commit: automatic –10 % on Part B.
- Part A answers that restate lecture content without adding analysis: 0 on the reasoning sub-criterion for that question.
- Benchmarks without repetition or without stating the number of runs: –10 % on Part C.
- "X is good for Y-type data" without explaining why the access pattern demands X: 0 on justification sub-criterion for A.5.
- Docker Compose file that requires manual steps beyond docker compose up --build: –10 % on Part B.

A note on AI tools

Using AI assistance for debugging, understanding error messages, or checking syntax is fine and does not need to be disclosed. Using it to draft answers to Part A or Part D, or to generate substantial portions of the report narrative, must be disclosed in the AI Use Disclosure appendix. The mark assigned to undisclosed AI content, if discovered, is zero for the affected section. The mark assigned to disclosed AI content is at the marker's discretion, but disclosure alone does not penalise you.

4. Required and Recommended Reading

The items marked [required] must appear in your bibliography and must be cited where relevant. The remaining items are provided to help you go further. A report that draws exclusively on lecture slides and Wikipedia will not reach the standards expected at this level of the degree.

4.1 Required References

1. [required] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media. — Chapters 5, 6, and 11 are directly relevant to this assignment.
2. [required] Gilbert, S. & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59.
3. [required] Abadi, D. J. (2012). Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. *IEEE Computer*, 45(2), 37–42.
4. [required] Chang, F. et al. (2006). Bigtable: A distributed storage system for structured data. *ACM TOCS*, 26(2), 1–26. — The architectural foundation of all column-family systems covered in Week 6.
5. [required] Robinson, I., Webber, J. & Eifrem, E. (2015). *Graph Databases: New Opportunities for Connected Data* (2nd ed.). O'Reilly Media. — Chapters 2 and 4.
6. [required] Vogels, W. (2009). Eventually consistent. *Communications of the ACM*, 52(1), 40–44.

4.2 Recommended Reading

1. DeCandia, G. et al. (2007). Dynamo: Amazon's highly available key-value store. *ACM SIGOPS SOSP*.
2. Lakshman, A. & Malik, P. (2010). Cassandra: A decentralized structured storage system. *ACM SIGOPS Operating Systems Review*, 44(2), 35–40.
3. Sadalage, P. J. & Fowler, M. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley.
4. Discord Engineering Blog (2023). Migrating Discord's Trillions of Messages from Cassandra to ScyllaDB. engineering.discord.com.
5. Neo4j, Inc. (2025). Neo4j Developer Documentation — Cypher Reference. neo4j.com/docs.
6. Apache Cassandra Project (2025). Cassandra 4.1 Documentation. cassandra.apache.org.
7. Week 1–7 lecture slides, Advanced Data Management, ECE — University of Thessaly, Spring 2026.

Final Word

The problems in this assessment are not academic exercises invented for a course. They are simplified versions of real engineering decisions made by the teams that build the infrastructure behind power grids, telecommunications networks, and financial systems — systems where a wrong data architecture has consequences measured in money, service outages, or safety. The best preparation you can bring to this work is not memorised syntax. It is the habit of asking, every time you make a decision: what am I trading off, and for

whom? That question will serve you throughout your career.